Web Scraping: A Beginner's Guide

Notes on Large-Scale Data Scraping

Sankalp Sharma

February 2026

Contents

1 What Is Web Scraping? 4
- 1.1 The HTTP Request/Response Cycle . . . 4
  - 1.1.1 What Happens When You Visit a URL . . . 4
  - 1.1.2 Request Methods . . . 4
  - 1.1.3 Headers, Cookies, and Sessions . . . 4
  - 1.1.4 Status Codes . . . 5
- 1.2 HTML Structure . . . 5
  - 1.2.1 Tags, Attributes, and the DOM . . . 5
  - 1.2.2 CSS Selectors . . . 6
- 1.3 The Browser vs. the Scraper . . . 6
- 1.4 Legal and Ethical Framework . . . 7
  - 1.4.1 Indian Legal Landscape . . . 7
  - 1.4.2 robots.txt and Terms of Service . . . 7
  - 1.4.3 Ethical Guidelines . . . 8
- 1.5 When Is Scraping Appropriate? . . . 8

2 Your First Scraper with Python 8
- 2.1 Setting Up `requests.Session` . . . 8
- 2.2 Parsing HTML with BeautifulSoup . . . 9
- 2.3 Extracting Structured Data from Tables . . . 11
- 2.4 Saving to CSV and JSON . . . 12
- 2.5 Complete Worked Example: Scraping a Public Directory . . . 12
  - 2.5.1 Handling Common Failures . . . 14

3 Prompt Engineering for Scraping with Claude Code 14
- 3.1 Role Prompting: Setting Context . . . 14
- 3.2 Schema-First Prompting . . . 15

3.3 Few-Shot Examples . . . 16
3.4 Iterative Refinement Workflow . . . 16
3.5 Debugging Prompts . . . 17
3.6 Anti-Patterns: What NOT to Do . . . 18
3.7 Reusable Prompt Templates . . . 18

4 Handling Complex Web Technologies 19
4.1 ASP.NET WebForms: The Stateful Beast . . . 19
4.2 CSRF Tokens and Session Management . . . 22
4.3 JavaScript-Rendered Pages . . . 22
4.4 ArcGIS REST API: Spatial Data at Scale . . . 23
4.5 AJAX/XHR Endpoints: Finding the Real Data Source . . . 25

5 Scaling Up 25
5.1 Why Async Matters . . . 25
5.2 Basic aiohttp Pattern . . . 26
5.3 Multi-Session Architecture . . . 27
5.4 Batch Processing with Periodic Flush . . . 28
5.5 Process-Level Parallelism with tmux . . . 29
5.6 Token Bucket Rate Limiting . . . 30
5.7 Adaptive Query Splitting . . . 30

6 Anti-Detection and CAPTCHAs 31
6.1 How Websites Detect Scrapers . . . 31
6.2 User-Agent Strings . . . 32
6.3 Full Browser Headers . . . 32
6.4 Playwright Stealth Configuration . . . 33
6.5 Human-Like Behavior . . . 34
6.6 Proxy Rotation . . . 34
6.7 CAPTCHA Solving with Tesseract OCR . . . 35
6.8 CAPTCHA Avoidance Strategies . . . 36
6.9 SSL Certificate Issues . . . 36

7 Building Robust Pipelines 37
7.1 Resumable Progress Tracking . . . 37
7.1.1 Approach 1: JSON Progress File . . . 37
7.1.2 Approach 2: SQLite with WAL Mode . . . 38
7.1.3 Approach 3: File-Existence Check . . . 38
7.2 Atomic File Writes . . . 39
7.3 Retry with Exponential Backoff . . . 39
7.3.1 Automatic Retries with urllib3 . . . 39

7.3.2 Manual Retry Loop 40
7.4 Circuit Breaker Pattern 40
7.5 Session Refresh Strategies 41
7.6 Data Storage Patterns 42
7.7 Canonical Records with History Merging 42

8 Putting It All Together **43**
8.1 The Standard Pipeline Template 43
8.2 Checklist for Starting a New Scraping Project 45
8.3 Common Mistakes and Debugging 46
8.4 Debugging Strategies 46
8.5 Resources for Further Learning 47

1 What Is Web Scraping?

1.1 The HTTP Request/Response Cycle

Every interaction on the web follows a simple pattern: your computer asks a server for something, and the server replies. When you type a URL into your browser and press Enter, a chain of events unfolds in milliseconds.

HTTP stands for HyperText Transfer Protocol. HTTPS is the encrypted version.

1.1.1 What Happens When You Visit a URL

1. Your browser resolves the domain name (e.g., `data.gov.in`) to an IP address via DNS.
2. It opens a TCP connection to that IP address on port 80 (HTTP) or 443 (HTTPS).
3. It sends an **HTTP request** containing a method, path, headers, and optionally a body.
4. The server processes the request and returns an **HTTP response** containing a status code, headers, and a body (usually HTML).
5. Your browser parses the HTML, discovers linked resources (CSS, JavaScript, images), and fires off additional requests for each one.

1.1.2 Request Methods

The two methods you will use most often when scraping:

- **GET** requests data from a server. The parameters travel in the URL itself (the query string after `?`). Every time you click a link or type a URL, your browser sends a GET request.
- **POST** sends data to a server in the request body. Login forms, search forms, and any action that changes state typically use POST.

1.1.3 Headers, Cookies, and Sessions

Headers are key-value metadata attached to both requests and responses. Three headers matter immediately for scraping:

Servers use `User-Agent` to decide what content to serve. Some block requests that lack a browser-like User-Agent.

- `User-Agent`: identifies the client software (e.g., `Mozilla/5.0 ...` for Chrome).
- `Cookie`: sends stored cookies back to the server so it can recognize returning visitors.

- `Content-Type`: tells the server how to interpret the body (e.g., form data or JSON).

Cookies are small pieces of data the server asks your browser to store and send back with subsequent requests. They enable sessions: the server issues a session cookie on your first visit, and every request carrying that cookie is recognized as part of the same session.

1.1.4 Status Codes

The server's response starts with a three-digit status code. Memorize these:

Code	Meaning	Scraper implication
200	OK	You got the page. Parse it.
301/302	Redirect	Follow the `Location` header.
403	Forbidden	The server refused your request.
404	Not Found	The URL is wrong or the page is gone.
429	Too Many Requests	You are being rate-limited. Back off.
500	Internal Server Error	Server-side problem. Retry later.

Warning

A 429 response means the server has detected that you are sending requests too quickly. If you ignore it and keep hammering the server, your IP address may be permanently blocked. Always respect rate limits.

1.2 HTML Structure

HTML is the skeleton of every web page. Understanding its structure is the single most important skill for scraping, because your job is to locate specific pieces of data inside an HTML document.

1.2.1 Tags, Attributes, and the DOM

An HTML document is a tree of nested **elements**. Each element is defined by a **tag** and may have **attributes**:

```
<!-- A small HTML fragment -->
<div class="employee-card" id="emp-101">
    <h3>Priya Sharma</h3>
    <span class="designation">Section Officer</span>
    <a href="/profile/101">View Profile</a>
</div>
```

In this fragment:

- `<div>`, `<h3>`, `<span>`, and `<a>` are tags.
- `class="employee-card"`, `id="emp-101"`, and `href="/profile/101"` are attributes.
- The text "Priya Sharma" is the **text content** of the `<h3>` element.

The **Document Object Model (DOM)** is the browser's in-memory representation of this tree. When we scrape, we build a similar tree using a parser like BeautifulSoup and then search through it.

Think of the DOM as a family tree. Every element has a parent, and may have children and siblings.

1.2.2 CSS Selectors

CSS selectors are patterns that identify elements. You already use them when writing CSS; they work identically in scraping libraries.

Selector	Matches
`div`	All `<div>` elements
`.employee-card`	Elements with `class="employee-card"`
`#emp-101`	The element with `id="emp-101"`
`div.employee-card h3`	`<h3>` inside a `<div class="employee-card">`

Tip

Use your browser's Developer Tools (F12 or Ctrl+Shift+I) to inspect elements on any page. Right-click an element, choose "Inspect," and you will see its HTML. You can also right-click the element in the Elements panel and select "Copy → Copy selector" to get a CSS selector for it.

1.3 The Browser vs. the Scraper

Your browser does far more than send HTTP requests. Understanding the gap between what a browser does and what a simple scraper does will save you hours of confusion.

When your scraper gets a blank page but the browser shows data, JavaScript rendering is almost always the cause.

Capability	Browser	`requests` library
Send HTTP requests	Yes	Yes
Receive HTML responses	Yes	Yes
Execute JavaScript	Yes	No
Render CSS / layout	Yes	No
Handle cookies automatically	Yes	Only with `Session`
Follow redirects	Yes	Yes (by default)
Load images, fonts, CSS files	Yes	No
Run AJAX/fetch calls	Yes	No

The key implication: if a website loads its data via JavaScript (using AJAX calls after the initial page load), the HTML returned by `requests.get()` will not contain that data. You will see an empty container or a loading spinner in the raw HTML. In such cases, you have two options:

1. Find the underlying API endpoint that the JavaScript calls (inspect Network tab in Developer Tools) and hit it directly. This is faster and preferred.
2. Use a browser automation tool like Selenium or Playwright that runs a real browser. This is slower but handles any JavaScript-rendered page.

1.4 Legal and Ethical Framework

Web scraping occupies a grey area in law. The legality depends on what you scrape, how you scrape it, and what you do with the data.

1.4.1 Indian Legal Landscape

IT Act 2000, Section 43 imposes penalties for "unauthorized access" to computer systems. The operative question is whether accessing publicly available web pages constitutes "unauthorized access." Courts have generally held that it does not, provided you do not bypass authentication or technical barriers.

OLX India Pvt. Ltd. v. Padawan Technologies (2020, Delhi High Court) is a landmark case. OLX sued Padawan for scraping its publicly listed classified ads. The Delhi HC held that scraping publicly available data did not amount to unauthorized access under Section 43, since the data was freely accessible to any visitor.

Digital Personal Data Protection Act, 2023 (DPDPA) applies when you collect, store, or process *personal data* (data that identifies a living individual). Scraping publicly available personal data does not automatically exempt you from DPDPA obligations.

National Data Sharing and Accessibility Policy (NDSAP) promotes open access to government data. Websites like https://data.gov.in publish datasets explicitly intended for public use.

1.4.2 robots.txt and Terms of Service

robots.txt is a file at the root of a website (e.g., https://example.com/robots.txt) that tells crawlers which paths they may and may not access. It is a convention, not a legal barrier, but violating robots.txt can be used as evidence of bad faith in legal proceedings.

Terms of Service (ToS) on many websites explicitly prohibit scraping. Violating ToS is a breach of contract. Read the ToS before scraping a commercial website.

1.4.3 Ethical Guidelines

1. **Rate-limit your requests.** Do not send more than one request per second unless you know the server can handle more.
2. **Do not scrape personal data** unless you have a lawful basis and a clear purpose.
3. **Identify yourself.** Set a descriptive `User-Agent` header.
4. **Cache aggressively.** Save responses to disk so you do not re-download pages.
5. **Attribute your sources.** Credit the original source if you publish scraped data.
6. **Stop if asked.** If a site operator contacts you and asks you to stop, stop.

1.5 When Is Scraping Appropriate?

Not every data collection problem requires scraping. Before writing a scraper, walk through this decision tree:

1. **Is there an official API?** Check the website's developer documentation. Always prefer an API.
2. **Is the data already available as a download?** Check https://data.gov.in first.
3. **Is the data public?** If the data requires login credentials you do not own, do not scrape it.
4. **Does robots.txt allow access?** Check `/robots.txt` on the target domain.
5. **Would your scraping overload the server?** Be especially gentle with small government portals.
6. **Does the Terms of Service prohibit scraping?** If yes, assess the legal risk before proceeding.

2 Your First Scraper with Python

2.1 Setting Up `requests.Session`

The `requests` library is the standard tool for making HTTP requests in Python. Use a `Session` object instead of bare `requests.get()` calls for three reasons: it persists cookies across requests, reuses TCP connections (faster), and lets you set default headers once.

A `Session` is like keeping the same browser tab open. Cookies, headers, and the connection carry over from one request to the next.

```python
import requests

# Create a session with browser-like headers
session = requests.Session()
session.headers.update({
    "User-Agent": "MyScraper/1.0 (research project; contact@example.com)"
})

# GET request
response = session.get("https://httpbin.org/get")
print(response.status_code)   # 200
print(response.headers["Content-Type"])  # application/json
print(response.text[:200])    # First 200 chars of response body

# POST request with form data
response = session.post(
    "https://httpbin.org/post",
    data={"username": "test", "query": "web scraping"}
)
print(response.json())  # Parse JSON response into a dict
```

Key attributes of the `Response` object:

Attribute	What it gives you
`response.status_code`	HTTP status code (int)
`response.text`	Response body as a string (decoded)
`response.content`	Response body as raw bytes
`response.json()`	Response body parsed as JSON (dict/list)
`response.headers`	Response headers (dict-like)
`response.url`	Final URL after redirects
`response.cookies`	Cookies set by this response

Tip

Always check `response.status_code` before parsing the body. Use `response.raise_for_status()` to raise an exception on 4xx/5xx responses so you catch failures early instead of silently parsing an error page.

2.2 Parsing HTML with BeautifulSoup

BeautifulSoup transforms an HTML string into a navigable tree of Python objects. The mental model has four steps:

Install with `pip install beautifulsoup4 lxml`. The `lxml` parser is faster than the default `html.parser`.

1. **Fetch**: get the HTML string using `requests`.

2. **Parse**: pass the string to BeautifulSoup to build a tree.
3. **Search**: use `find()`, `find_all()`, or `select()` to locate elements.
4. **Extract**: pull out text content (`.text`) or attribute values (`["href"]`).

```python
from bs4 import BeautifulSoup

html = """
<html>
<body>
  <div class="directory">
    <div class="entry">
      <h3 class="name">Amit Verma</h3>
      <span class="dept">Revenue Department</span>
      <a href="/profile/201">View</a>
    </div>
    <div class="entry">
      <h3 class="name">Sunita Rao</h3>
      <span class="dept">Education Department</span>
      <a href="/profile/202">View</a>
    </div>
  </div>
</body>
</html>
"""

# Step 2: Parse
soup = BeautifulSoup(html, "lxml")

# Step 3a: find() returns the FIRST matching element
first_name = soup.find("h3", class_="name")
print(first_name.text)  # "Amit Verma"

# Step 3b: find_all() returns ALL matching elements
all_names = soup.find_all("h3", class_="name")
for tag in all_names:
    print(tag.text)  # "Amit Verma", then "Sunita Rao"

# Step 3c: select() uses CSS selectors
links = soup.select("div.entry a[href^='/profile']")
for link in links:
    print(link["href"])  # "/profile/201", then "/profile/202"
```

2.3 Extracting Structured Data from Tables

HTML tables are the most common structured data format on government websites. The pattern: find the table, extract headers from `<th>` tags, then extract each row's cells from `<td>` tags.

```
from bs4 import BeautifulSoup

html_table = """
<table id="budget-data">
  <thead>
    <tr>
      <th>Department</th>
      <th>Allocation (Cr)</th>
      <th>Expenditure (Cr)</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Education</td><td>1,12,899</td><td>98,450</td></tr>
    <tr><td>Health</td><td>89,155</td><td>76,230</td></tr>
    <tr><td>Defence</td><td>5,93,537</td><td>5,41,200</td></tr>
  </tbody>
</table>
"""

soup = BeautifulSoup(html_table, "lxml")
table = soup.find("table", id="budget-data")

# Extract headers
headers = [th.text.strip() for th in table.find("thead").find_all("th")]

# Extract rows as list of dicts
rows = []
for tr in table.find("tbody").find_all("tr"):
    cells = [td.text.strip() for td in tr.find_all("td")]
    rows.append(dict(zip(headers, cells)))

print(rows)
# [{'Department': 'Education', 'Allocation (Cr)': '1,12,899', ...}]
```

Tip

Always use `.text.strip()` when extracting text from HTML elements. Raw text often contains leading/trailing whitespace and newline characters that will pollute your data.

2.4 Saving to CSV and JSON

```
import csv, json

# CSV with DictWriter
with open("budget_data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=rows[0].keys())
    writer.writeheader()
    writer.writerows(rows)

# JSON
with open("budget_data.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2, ensure_ascii=False)
```

Always specify `encoding="utf-8"` and `newline=""` when writing CSV files. The empty `newline` argument prevents double-spaced rows on Windows.

2.5 Complete Worked Example: Scraping a Public Directory

We will now build a complete scraper from scratch. Our target is a fictional state government employee directory at `https://example-gov-directory.in/employees`.

We use a fictional URL here. When you adapt this for a real site, inspect the actual HTML structure first using Developer Tools (F12).

Warning

Before scraping any real website, always: (1) Check `/robots.txt` for disallowed paths. (2) Read the Terms of Service. (3) Verify the data is not available through an API or bulk download. (4) Ensure you are not collecting personal data without a lawful basis under DPDPA 2023.

```
"""Scraper for a public government employee directory."""
import time, csv, requests
from bs4 import BeautifulSoup

BASE_URL = "https://example-gov-directory.in"
START_URL = f"{BASE_URL}/employees"

def create_session():
```

```
    """Create a session with polite, identifiable headers."""
    s = requests.Session()
    s.headers.update({
        "User-Agent": "GovDirectoryScraper/1.0 (research; you@example.com)
    "
    })
    return s

def scrape_page(session, url):
    """Scrape one page. Returns (list_of_dicts, next_url_or_None)."""
    response = session.get(url)
    response.raise_for_status()
    soup = BeautifulSoup(response.text, "lxml")
    table = soup.find("table", class_="emp-table")

    headers = [th.text.strip() for th in table.find("thead").find_all("th
    ")]
    employees = []
    for tr in table.find("tbody").find_all("tr"):
        cells = [td.text.strip() for td in tr.find_all("td")]
        employees.append(dict(zip(headers, cells)))

    next_link = soup.select_one("div.pagination a")
    next_url = None
    if next_link and "Next" in next_link.text:
        next_url = BASE_URL + next_link["href"]
    return employees, next_url

def scrape_all_pages(session):
    """Follow pagination and collect all employees."""
    all_employees, url, page_num = [], START_URL, 1
    while url:
        print(f"Scraping page {page_num}: {url}")
        employees, url = scrape_page(session, url)
        all_employees.extend(employees)
        page_num += 1
        time.sleep(1)  # Polite delay
    return all_employees

if __name__ == "__main__":
    session = create_session()
    employees = scrape_all_pages(session)
    with open("employees.csv", "w", newline="", encoding="utf-8") as f:
```

```
        writer = csv.DictWriter(f, fieldnames=employees[0].keys())
        writer.writeheader()
        writer.writerows(employees)
    print(f"Saved {len(employees)} records to employees.csv")
```

2.5.1 Handling Common Failures

```
import time, requests

def scrape_page_robust(session, url, max_retries=3):
    """Scrape a page with retry logic for transient failures."""
    for attempt in range(max_retries):
        response = session.get(url)
        if response.status_code == 200:
            return parse_table(response.text)
        elif response.status_code == 429:
            wait = 2 ** attempt * 5  # 5s, 10s, 20s
            print(f"Rate limited. Waiting {wait}s before retry.")
            time.sleep(wait)
        elif response.status_code >= 500:
            print(f"Server error {response.status_code}. Retrying...")
            time.sleep(5)
        else:
            response.raise_for_status()
    raise Exception(f"Failed after {max_retries} retries: {url}")
```

Tip

Use exponential backoff when retrying after rate limits. Waiting 5, 10, 20 seconds (doubling each time) gives the server time to recover. The formula `2 ** attempt * base_wait` makes this easy to implement.

3 Prompt Engineering for Scraping with Claude Code

AI coding assistants can write scrapers faster than you can type import statements, but only if you give them the right instructions. A vague prompt produces vague code. A precise prompt produces production-ready code.

This chapter assumes you have Claude Code installed and authenticated. If not, see https://docs.anthropic.com/claude-code.

3.1 Role Prompting: Setting Context

Role prompting front-loads domain expertise into the conversation. When Claude Code knows *what kind* of scraping you are doing, it makes better choices about libraries,

error handling, and edge cases.

```
You are an expert web scraper specializing in Indian government
portals. You have deep experience with:
- ASP.NET WebForms (ViewState, EventValidation, postback chains)
- Tesseract OCR for solving simple CAPTCHAs on gov.in sites
- Session management with cookies across multi-step form workflows
- BeautifulSoup and requests for stateful scraping

All code should use Python 3.10+, type hints, and logging.
```

Role prompts are especially useful for government portals, which use frameworks and patterns rarely seen in modern web development.

3.2 Schema-First Prompting

Define what you want *before* asking for the code that gets it. Schema-first prompting forces you to think about your output structure, and it gives Claude Code an unambiguous target.

```
I need to scrape property tax records from the municipal corporation
portal at https://propertytax.example.gov.in/search.

Input:
- URL: https://propertytax.example.gov.in/search
- Search parameter: ward number (string, e.g., "042")

Output: JSON array where each record matches this schema:
{
  "property_id": "string -- unique municipal ID",
  "owner_name": "string -- full name as displayed",
  "ward_number": "string -- 3-digit ward code",
  "area_sqft": "float -- built-up area in square feet",
  "annual_tax": "float -- amount in INR",
  "payment_status": "string -- enum: paid | unpaid | partial",
  "last_payment_date": "string -- ISO 8601 date or null"
}

Write a scraper that takes a ward number and returns all records.
```

Tip

Defining the schema first catches misunderstandings early. If you ask for "property records" without specifying fields, you might get a scraper that extracts owner names but misses payment status, forcing a rewrite.

3.3 Few-Shot Examples

When the HTML structure is unusual, show Claude Code what you mean with concrete input-output pairs.

```
Given this HTML table row from the land records portal:

<tr class="odd">
  <td>2024-001</td>
  <td>Ramesh Kumar S/O Vijay Kumar</td>
  <td>12-A/3</td>
  <td>2.5 Hectare</td>
  <td><span class="status-green">Registered</span></td>
</tr>

I want this output:
{
  "record_id": "2024-001",
  "owner": "Ramesh Kumar S/O Vijay Kumar",
  "plot_number": "12-A/3",
  "area_hectares": 2.5,
  "status": "Registered"
}

Note: "Hectare" should be stripped, leaving only the numeric value
as a float. The status text should be extracted without the span.

Now write a function parse_row(row: Tag) -> dict that handles all
rows, including edge cases like missing fields.
```

Few-shot examples are the single most effective way to communicate parsing logic. One example eliminates pages of written specification.

3.4 Iterative Refinement Workflow

Do not ask for everything at once. Build scrapers in layers, testing each layer before adding the next.

Step 1: Basic extraction.

```
Write a Python scraper using requests and BeautifulSoup that:
1. Fetches https://records.example.gov.in/search?district=karnal
2. Extracts all rows from the HTML table with id="recordsTable"
3. Prints each row as a dictionary
Keep it simple -- no error handling yet, just get the data.
```

Step 2: Resilience.

```
The basic scraper works. Now add:
- Retry logic with exponential backoff (3 retries, starting at 2s)
- Timeout of 30 seconds per request
- Proper HTTP error handling (log and skip 404s, retry 500s)
- Save partial results if the script crashes mid-run
```

Step 3: Resumability.

```
This scraper needs to process 700 districts. Add:
- A progress file (progress.json) tracking completed districts
- On startup, load progress file and skip completed districts
- After each district completes, update the progress file
```

Step 4: Performance.

```
The scraper works but takes 6 hours for 700 districts. Convert it
to async using aiohttp with:
- Concurrency limit of 10 simultaneous requests
- Same retry/backoff logic as the sync version
- A rate limiter: max 20 requests per second
```

3.5 Debugging Prompts

When a scraper fails, include: what you expected, what happened, and the raw evidence.

```
This scraper returns an empty list but no errors. HTTP status is 200.
- URL: https://registry.example.gov.in/data
- Response body contains: <div id="results"></div> (empty div)
- When I open the same URL in a browser, I see 50 rows of data
- The page has several <script> tags referencing React
```

```
Why is the div empty in my requests response?
```

```
My ASP.NET form scraper gets "Invalid postback or callback argument"
on the second dropdown selection. Here is my POST data:

__VIEWSTATE: [base64 blob, 4KB]
__EVENTVALIDATION: [base64 blob]
__EVENTTARGET: ctl00$ContentPlaceHolder1$ddlTehsil
ctl00$ContentPlaceHolder1$ddlDistrict: 05
ctl00$ContentPlaceHolder1$ddlTehsil: 02

The first POST (selecting district) works fine. The error happens
when I POST to select the tehsil. Am I missing a hidden field?
```

3.6 Anti-Patterns: What NOT to Do

Bad Prompt	Better Prompt
"Scrape this website"	"Extract all product names and prices from the table on [URL]. Output as CSV."
"Build a complete scraping pipeline with async, proxies, CAPTCHA solving, and database storage"	"Write a basic scraper for this URL first. We will add features incrementally."
"My scraper doesn't work, fix it"	"My scraper returns empty results. Status is 200. Here is the HTML snippet and my parsing code: [code]"

Warning

The single most common anti-pattern is not providing the HTML. Claude Code cannot guess what the page structure looks like unless you show it. Always paste a representative HTML snippet (even 10–20 lines) into your prompt.

3.7 Reusable Prompt Templates

Template 1: Basic Table Scraper

```
Write a Python scraper (requests + BeautifulSoup) that:
1. Fetches [URL]
2. Finds the table [identified by: id/class/position]
3. Extracts these columns: [col1, col2, col3, ...]
4. Returns a list of dicts with keys: [key1, key2, key3, ...]
```

```
5. Saves to [filename].csv
Handle: missing cells (use None), merged rows, Unicode text.
```

Template 2: Form Submission Scraper

```
Write a Python scraper that submits a search form:
1. GET [form URL] to load the page and extract hidden tokens
2. POST to [action URL] with these fields:
   - [field1]: [value or description]
   - Include all hidden fields from step 1
3. Parse the results page and extract: [target data]
4. Handle pagination if results span multiple pages
The form uses [ASP.NET WebForms / Django CSRF / standard HTML].
Here is a sample of the form HTML: [paste snippet]
```

Template 3: Debugging a Failing Scraper

```
My scraper for [URL] is failing. Diagnostic info:
Symptom: [empty results / error / wrong data]
Expected: [what should happen]
HTTP status: [status code]
Response body sample: [first 500 chars or relevant snippet]
My code: [paste the relevant function]
Error traceback (if any): [paste traceback]
What is wrong and how do I fix it?
```

4 Handling Complex Web Technologies

4.1 ASP.NET WebForms: The Stateful Beast

This chapter covers the technologies that make scraping hard. If a site works with a simple GET request, you do not need any of this.

ASP.NET WebForms breaks a fundamental assumption of HTTP: that requests are stateless. Every interaction with a WebForms page is a **postback**, a POST request carrying the entire page state back to the server.

ASP.NET WebForms was introduced in 2002 and is still used by hundreds of Indian government portals.

Why is this hard to scrape? Three reasons:

1. **ViewState**: A Base64-encoded blob (often 10–50 KB) stored in a hidden field containing serialized page state. You must extract it from each response and include it in the next POST.

2. **EventValidation**: A whitelist of allowed postback events. POST an event not in this list and the server returns "Invalid postback or callback argument."

3. **Cascading dropdowns**: Selecting a district triggers a postback that populates the tehsil dropdown, which triggers another postback for villages. Each step requires fresh tokens.

```
import requests
from bs4 import BeautifulSoup
from typing import Optional

class WebFormsScraper:
    """Manages ASP.NET WebForms state across postback chains."""

    TOKEN_FIELDS = [
        "__VIEWSTATE", "__VIEWSTATEGENERATOR",
        "__EVENTVALIDATION", "__VIEWSTATEENCRYPTED",
    ]

    def __init__(self, base_url: str):
        self.session = requests.Session()
        self.base_url = base_url
        self.tokens: dict[str, str] = {}

    def _extract_tokens(self, html: str) -> dict[str, str]:
        soup = BeautifulSoup(html, "html.parser")
        tokens = {}
        for field_name in self.TOKEN_FIELDS:
            tag = soup.find("input", {"name": field_name})
            if tag and tag.get("value"):
                tokens[field_name] = tag["value"]
        return tokens

    def get_page(self, url: Optional[str] = None) -> BeautifulSoup:
        """Initial GET to load the page and capture tokens."""
        resp = self.session.get(url or self.base_url)
        resp.raise_for_status()
        self.tokens = self._extract_tokens(resp.text)
        return BeautifulSoup(resp.text, "html.parser")

    def postback(self, event_target: str, form_data: dict) ->
    BeautifulSoup:
        """Submit a postback with fresh tokens + form data."""
        payload = {**self.tokens}
        payload["__EVENTTARGET"] = event_target
        payload["__EVENTARGUMENT"] = ""
```

```
        payload.update(form_data)
        resp = self.session.post(self.base_url, data=payload)
        resp.raise_for_status()
        self.tokens = self._extract_tokens(resp.text)
        return BeautifulSoup(resp.text, "html.parser")
```

Cascading dropdown example (Haryana Jamabandi):

```
def scrape_jamabandi(district: str, tehsil: str, village: str):
    """Walk the four-step postback chain on Jamabandi."""
    scraper = WebFormsScraper(
        "https://jamabandi.nic.in/land-records/NakalRecord"
    )
    scraper.get_page()  # Load page, get initial tokens

    # Select district -> populates tehsil dropdown
    scraper.postback(
        "ctl00$ContentPlaceHolder1$ddlDistrict",
        {"ctl00$ContentPlaceHolder1$ddlDistrict": district},
    )
    # Select tehsil -> populates village dropdown
    scraper.postback(
        "ctl00$ContentPlaceHolder1$ddlTehsil",
        {"ctl00$ContentPlaceHolder1$ddlDistrict": district,
         "ctl00$ContentPlaceHolder1$ddlTehsil": tehsil},
    )
    # Select village -> results page
    soup = scraper.postback(
        "ctl00$ContentPlaceHolder1$ddlVillage",
        {"ctl00$ContentPlaceHolder1$ddlDistrict": district,
         "ctl00$ContentPlaceHolder1$ddlTehsil": tehsil,
         "ctl00$ContentPlaceHolder1$ddlVillage": village},
    )
    return soup.find("table", {"id": "ContentPlaceHolder1_gvData"})
```

Tip

When debugging WebForms postbacks, use your browser's DevTools Network tab to capture a successful manual submission. Compare the form fields your code sends against what the browser sends. The difference is almost always a missing hidden field.

4.2 CSRF Tokens and Session Management

CSRF tokens are one-time (or per-session) secrets embedded in HTML forms. The extraction pattern is always the same: GET the page, find the token, include it in the POST.

CSRF stands for Cross-Site Request Forgery. The token proves that the POST originated from the site's own form.

```
def submit_with_csrf(url: str, search_term: str) -> str:
    """Generic CSRF-aware form submission."""
    session = requests.Session()
    resp = session.get(url)
    soup = BeautifulSoup(resp.text, "html.parser")

    # Common CSRF field names across frameworks
    csrf_names = ["csrfmiddlewaretoken", "_csrf", "authenticity_token"]
    csrf_token, csrf_field = None, None
    for name in csrf_names:
        tag = soup.find("input", {"name": name})
        if tag:
            csrf_token = tag["value"]
            csrf_field = name
            break

    if not csrf_token:
        raise ValueError("No CSRF token found on the page")

    payload = {csrf_field: csrf_token, "query": search_term}
    result = session.post(url, data=payload)
    result.raise_for_status()
    return result.text
```

Warning

Some sites place the CSRF token in a response header (e.g., `X-CSRF-Token`) or in a `<meta>` tag rather than a form field. If the hidden input approach fails, check the page's `<head>` section and response headers.

4.3 JavaScript-Rendered Pages

If you `requests.get()` a URL and the data is missing, the page likely requires JavaScript to render. You have two options: find the underlying API (preferred) or use Playwright.

Use requests (no JS engine)	Use Playwright (full browser)
Data is in the initial HTML	Data loads via JS after page load
Data comes from a discoverable XHR endpoint	No API; data rendered by a JS framework
Performance matters	You need to click, scroll, interact

Playwright supports Chromium, Firefox, and WebKit. For scraping, Chromium in headless mode is standard.

```python
from playwright.sync_api import sync_playwright

def scrape_js_rendered_table(url: str) -> list[dict]:
    """Use Playwright when data only appears after JS execution."""
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()
        page.goto(url)
        page.wait_for_selector(
            "table.data-table tbody tr", timeout=15_000
        )
        rows = page.eval_on_selector_all(
            "table.data-table tbody tr",
            """rows => rows.map(row => {
                const cells = row.querySelectorAll('td');
                return {
                    name: cells[0]?.textContent.trim(),
                    value: cells[1]?.textContent.trim(),
                    date: cells[2]?.textContent.trim()
                };
            })"""
        )
        browser.close()
        return rows
```

Tip

Before reaching for Playwright, open DevTools → Network → filter by XHR/Fetch. Reload the page. Often the data comes from a clean JSON API that you can call directly with `requests`, bypassing JS rendering entirely.

4.4 ArcGIS REST API: Spatial Data at Scale

Many government geospatial datasets are served through ArcGIS REST services. These are structured APIs with query endpoints, not web pages to scrape.

ArcGIS services are the backbone of government GIS portals across India and globally.

```python
import requests

def discover_service(service_url: str) -> dict:
    """Query an ArcGIS REST service for metadata."""
    resp = requests.get(service_url, params={"f": "json"})
    metadata = resp.json()
    print(f"Name: {metadata['name']}")
    print(f"Geometry: {metadata['geometryType']}")
    print(f"Max records: {metadata['maxRecordCount']}")
    for field in metadata["fields"]:
        print(f"  {field['name']:30s} ({field['type']})")
    return metadata

def query_arcgis(service_url, where="1=1", offset=0, count=1000):
    """Query an ArcGIS feature layer with pagination."""
    params = {
        "where": where, "outFields": "*", "f": "geojson",
        "resultOffset": offset, "resultRecordCount": count,
        "returnGeometry": "true",
    }
    resp = requests.get(f"{service_url}/query", params=params)
    resp.raise_for_status()
    return resp.json()
```

Pagination strategies:

1. **Offset-based**: Increment `resultOffset` until `exceededTransferLimit` is false.
2. **ObjectID-based**: Query for min/max ObjectID, then request ranges. Works on all servers.
3. **Spatial filter**: Divide by ward/district attribute. Necessary for very large layers (300K+).

For most use cases, the `esridump` library handles pagination automatically:

```python
from esridump.dumper import EsriDumper
import json

dumper = EsriDumper(service_url)
features = list(dumper)  # Handles pagination internally

with open("output.geojson", "w") as f:
```

```
    json.dump({"type": "FeatureCollection", "features": features}, f)
print(f"Downloaded {len(features)} features")
```

4.5 AJAX/XHR Endpoints: Finding the Real Data Source

Many modern websites display data through JavaScript that fetches from a backend API. If you can find and replicate these API calls, you skip the browser entirely.

Step-by-step discovery:

1. Open DevTools (`F12`). Go to the **Network** tab. Check **Preserve log**.
2. Filter by **Fetch/XHR**.
3. Perform the action that loads data (click search, scroll, etc.).
4. Find the request that returns your data (check the Response tab).
5. Right-click → **Copy** → **Copy as cURL**.
6. Convert the cURL command to Python using https://curlconverter.com.

```
def fetch_from_discovered_api():
    """Converted from a cURL command captured in DevTools."""
    api_url = "https://dashboard.example.gov.in/api/v2/stats"
    headers = {
        "Accept": "application/json",
        "Referer": "https://dashboard.example.gov.in/districts",
        "X-Requested-With": "XMLHttpRequest",
    }
    params = {"state_code": "06", "year": "2024", "format": "json"}
    resp = requests.get(api_url, headers=headers, params=params)
    resp.raise_for_status()
    return resp.json()["districts"]
```

This approach is almost always preferable to browser automation. API responses are structured JSON, smaller than full HTML pages, and faster to process.

5 Scaling Up

5.1 Why Async Matters

Synchronous scraping is the bottleneck you did not know you had. When your code sends a request and waits for the response before sending the next one, roughly 95% of execution time is spent idle, waiting on the network.

Most beginners never question the one-request-at-a-time model until they hit a six-figure target.

Consider the arithmetic. A government portal responds in 500ms on average. Synchronous scraping gives you 2 requests per second. If you need 1 million records:

```
1,000,000 records / 2 req/s = 500,000 seconds = 5.8 days
```

Asynchronous scraping with 30 concurrent requests:

```
1,000,000 records / 30 req/s = 33,333 seconds = 9.3 hours
```

Same machine, same IP, same thread. The difference is that async code sends the next request *while waiting* for the previous one to return.

Tip

Async is not parallelism. It runs on a single thread and a single CPU core. It achieves speed by overlapping network wait times, not by doing more computation. This means async adds almost zero CPU overhead.

The key abstraction is the **event loop**, an async runtime that manages all in-flight tasks. Think of a switchboard operator who connects calls, and while one caller is on hold, immediately connects the next.

5.2 Basic aiohttp Pattern

The `aiohttp` library is the async equivalent of `requests`.

`aiohttp` supports connection pooling, keep-alive, and streaming out of the box.

```python
import asyncio, aiohttp, random

async def fetch(session, url, semaphore, max_retries=4):
    """Fetch a single URL with concurrency limiting and backoff."""
    async with semaphore:
        for attempt in range(max_retries):
            try:
                async with session.get(url,
                    timeout=aiohttp.ClientTimeout(total=30)) as resp:
                    if resp.status == 200:
                        return await resp.json()
                    elif resp.status == 429:
                        wait = 2 ** attempt + random.uniform(0, 1)
                        await asyncio.sleep(wait)
                    else:
                        resp.raise_for_status()
```

```
            except (aiohttp.ClientError, asyncio.TimeoutError):
                if attempt == max_retries - 1:
                    return None
                await asyncio.sleep(2 ** attempt + random.uniform(0, 1))

async def scrape_all(urls):
    """Scrape URLs with bounded concurrency."""
    semaphore = asyncio.Semaphore(30)
    connector = aiohttp.TCPConnector(limit=30, ttl_dns_cache=300)
    async with aiohttp.ClientSession(connector=connector) as session:
        tasks = [fetch(session, url, semaphore) for url in urls]
        results = await asyncio.gather(*tasks)
    return [r for r in results if r is not None]

results = asyncio.run(scrape_all(url_list))
```

The `Semaphore(30)` allows at most 30 requests in-flight at any moment.

Without a semaphore, `gather` on 100k URLs would open 100k connections simultaneously, crashing your machine.

5.3 Multi-Session Architecture

Some servers rate-limit per session rather than per IP. A single session hitting 100 req/s triggers throttling. But 20 sessions each at 5 req/s look like 20 separate users while delivering 100 req/s aggregate.

This pattern was discovered scraping the GHMC (Greater Hyderabad) property database, hence the "GHMC pattern" name.

```
from itertools import cycle

async def init_sessions(n=20, base_url="https://portal.example.gov"):
    """Create N independent sessions with their own cookies."""
    sessions = []
    for i in range(n):
        connector = aiohttp.TCPConnector(limit=10)
        session = aiohttp.ClientSession(connector=connector)
        async with session.get(base_url) as resp:
            await resp.text()  # Triggers Set-Cookie headers
        sessions.append(session)
        await asyncio.sleep(0.5)  # Stagger creation
    return sessions

async def scrape_multi_session(record_ids):
    """Round-robin requests across multiple sessions."""
    sessions = await init_sessions(n=20)
    semaphore = asyncio.Semaphore(60)
```

```
    pool = cycle(sessions)

    async def fetch_one(rid):
        s = next(pool)
        async with semaphore:
            url = f"https://portal.example.gov/api/record/{rid}"
            async with s.get(url) as resp:
                return await resp.json() if resp.status == 200 else None

    results = await asyncio.gather(*[fetch_one(r) for r in record_ids])
    for s in sessions:
        await s.close()
    return results
```

Warning

Session initialization must be staggered. Creating 20 sessions in a burst from the same IP in under a second is itself a detection signal. Space them out by at least 0.3–0.5 seconds.

5.4 Batch Processing with Periodic Flush

Holding a million records in memory until the job finishes is a recipe for data loss.

A crash at record 800,000 with no flushing means you lose everything and start over.

```
import json, time, os

class BatchWriter:
    """Buffer records, flush to numbered JSONL chunk files."""
    def __init__(self, output_dir, prefix="records",
                 batch_size=5000, flush_interval=60):
        self.output_dir = output_dir
        self.prefix = prefix
        self.batch_size = batch_size
        self.flush_interval = flush_interval
        self.buffer, self.chunk_index = [], 0
        self.last_flush = time.time()
        self.total_written = 0
        os.makedirs(output_dir, exist_ok=True)

    def add(self, record):
        self.buffer.append(record)
        if (len(self.buffer) >= self.batch_size or
            time.time() - self.last_flush >= self.flush_interval):
```

```python
            self.flush()

    def flush(self):
        if not self.buffer:
            return
        filename = f"{self.prefix}_{self.chunk_index:05d}.jsonl"
        filepath = os.path.join(self.output_dir, filename)
        with open(filepath, "w") as f:
            for record in self.buffer:
                f.write(json.dumps(record, ensure_ascii=False) + "\n")
        self.total_written += len(self.buffer)
        self.buffer, self.chunk_index = [], self.chunk_index + 1
        self.last_flush = time.time()

    def finalize(self):
        self.flush()
```

5.5 Process-Level Parallelism with tmux

When you have *independent targets* (different websites with separate rate limits), process-level parallelism is simpler than async.

`tmux` sessions survive SSH disconnections, which matters for multi-day jobs on remote servers.

```bash
#!/bin/bash
# launch_scrapers.sh
SESSION="scrape"
tmux new-session -d -s $SESSION
targets=("portal_a" "portal_b" "portal_c" "portal_d")
for i in "${!targets[@]}"; do
    target="${targets[$i]}"
    if [ $i -eq 0 ]; then
        tmux rename-window -t $SESSION "$target"
    else
        tmux new-window -t $SESSION -n "$target"
    fi
    tmux send-keys -t $SESSION:"$target" \
        "python scrape_${target}.py >> logs/${target}.log 2>&1" Enter
done
echo "Attach with: tmux attach -t $SESSION"
```

Tip

Use `tmux` when targets are independent. Use `async` when hitting a single target with many requests. The two compose well: multiple async scrapers, one per tmux window.

5.6 Token Bucket Rate Limiting

Fixed `sleep()` calls waste time. The token bucket algorithm maintains an average rate while allowing short bursts.

A 0.5s sleep enforces exactly 2 req/s, even if the server would happily accept short bursts of 10.

```
import asyncio, time

class RateLimiter:
    """Token bucket rate limiter for async code."""
    def __init__(self, rate=10.0, burst=20):
        self.rate = rate          # Tokens added per second
        self.burst = burst        # Maximum bucket capacity
        self.tokens = burst       # Start full
        self.last_refill = time.monotonic()
        self._lock = asyncio.Lock()

    async def acquire(self):
        async with self._lock:
            now = time.monotonic()
            elapsed = now - self.last_refill
            self.tokens = min(self.burst,
                              self.tokens + elapsed * self.rate)
            self.last_refill = now
            if self.tokens < 1.0:
                wait = (1.0 - self.tokens) / self.rate
                await asyncio.sleep(wait)
                self.tokens = 0.0
            else:
                self.tokens -= 1.0

limiter = RateLimiter(rate=30, burst=50)
```

5.7 Adaptive Query Splitting

Some APIs cap results per query. If your query matches 10,000 records but only the first 500 return, you silently lose data. Adaptive splitting detects this and narrows the search.

Property databases often cap at 500 or 1000 results per search.

```python
import string

async def adaptive_query(session, endpoint, base_params,
                         semaphore, max_results=500):
    """Try broad query, split by letter if results are capped."""
    data = await fetch(session, endpoint, base_params, semaphore)
    if data and len(data) < max_results:
        return data  # Full result set fits

    results = []
    for letter in string.ascii_uppercase:
        params = {**base_params, "name_starts_with": letter}
        data = await fetch(session, endpoint, params, semaphore)
        if data is None:
            continue
        if len(data) < max_results:
            results.extend(data)
        else:
            # Split further by two-letter prefix
            for second in string.ascii_uppercase:
                params2 = {**base_params,
                           "name_starts_with": letter + second}
                data2 = await fetch(session, endpoint, params2, semaphore)

                if data2:
                    results.extend(data2)
    return results
```

6 Anti-Detection and CAPTCHAs

6.1 How Websites Detect Scrapers

Government portals check 1–2 signals. Commercial sites check dozens.

Detection works by comparing your requests against the profile of a real browser. Four main categories:

HTTP headers. A real Chrome browser sends ∼15 headers. Python's `requests` sends 2. The mismatch is trivially detectable.

Request patterns. A request every exactly 1.000 seconds for 4 hours is not human behavior.

Browser fingerprinting. JavaScript checks `navigator.webdriver` (true in automation tools), empty `navigator.plugins`, and missing `window.chrome`.

IP reputation. Datacenter IP ranges (AWS, GCP) are catalogued and flagged. Residential IPs are trusted.

Signal	Government Portals	Commercial Sites
User-Agent check	Sometimes	Always
Full header validation	Rarely	Often
Request pattern analysis	Rarely	Always
Browser fingerprinting	Never	Common
IP reputation	Rarely	Always
CAPTCHA	Sometimes	Often

6.2 User-Agent Strings

```
# Desktop Chrome UA (update periodically)
CHROME_UA = (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
    "AppleWebKit/537.36 (KHTML, like Gecko) "
    "Chrome/120.0.0.0 Safari/537.36"
)

# Mobile UA -- useful for accessing mobile APIs
MOBILE_UA = (
    "Mozilla/5.0 (Linux; Android 14; Pixel 8) "
    "AppleWebKit/537.36 (KHTML, like Gecko) "
    "Chrome/120.0.0.0 Mobile Safari/537.36"
)

session = requests.Session()
session.headers["User-Agent"] = CHROME_UA
```

Some APIs check User-Agent to decide which response format to return. A mobile UA sometimes unlocks simpler JSON endpoints.

6.3 Full Browser Headers

```
BROWSER_HEADERS = {
    "User-Agent": CHROME_UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;"
              "q=0.9,image/avif,image/webp,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Accept-Encoding": "gzip, deflate, br",
    "Connection": "keep-alive",
    "Upgrade-Insecure-Requests": "1",
    "Sec-Fetch-Dest": "document",
    "Sec-Fetch-Mode": "navigate",
    "Sec-Fetch-Site": "none",
    "Sec-Fetch-User": "?1",
}
```

The `Sec-Fetch-*` headers were introduced by browsers specifically to help servers distinguish navigation from programmatic requests.

```
# ESRI/ArcGIS servers require Referer and Origin headers
ESRI_HEADERS = {
    **BROWSER_HEADERS,
    "Referer": "https://gis.example.gov/portal/apps/webappviewer/",
    "Origin": "https://gis.example.gov",
}
```

Warning

ESRI ArcGIS servers validate the `Referer` and `Origin` headers against a whitelist. Requests without these headers return 403 Forbidden. Inspect the browser's network tab to capture correct values.

6.4 Playwright Stealth Configuration

Playwright's headless mode is detectable by default. Stealth patches close the most common detection vectors.

```
from playwright.async_api import async_playwright

STEALTH_SCRIPT = """
Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
window.chrome = {runtime: {}, loadTimes: function(){}, csi: function(){}};

Object.defineProperty(navigator, 'plugins', {
    get: () => [
        {name: 'Chrome PDF Plugin', filename: 'internal-pdf-viewer'},
        {name: 'Chrome PDF Viewer', filename: '
    mhjfbmdgcfjbbpaeojofohoefgiehjai'},
        {name: 'Native Client', filename: 'internal-nacl-plugin'}
    ]
});
Object.defineProperty(navigator, 'languages', {
    get: () => ['en-US', 'en']
});
"""

async def create_stealth_browser():
    pw = await async_playwright().start()
    browser = await pw.chromium.launch(headless=True)
    context = await browser.new_context(
        viewport={"width": 1920, "height": 1080},
        locale="en-US",
        timezone_id="Asia/Kolkata",
        geolocation={"longitude": 77.2090, "latitude": 28.6139},
```

```
        permissions=["geolocation"],
        user_agent=CHROME_UA,
    )
    await context.add_init_script(STEALTH_SCRIPT)
    page = await context.new_page()
    return pw, browser, page
```

6.5 Human-Like Behavior

For commercial sites with behavioral analysis, simulate human patterns.

Government portals almost never employ behavioral analysis. Reserve these techniques for Cloudflare/Akamai-protected sites.

```
import random

def human_delay(low=0.5, mid=1.5, high=4.0):
    """Triangular distribution -- more realistic than uniform."""
    return random.triangular(low, high, mid)

async def human_scroll(page):
    """Scroll in random increments before interacting."""
    for _ in range(random.randint(2, 5)):
        await page.mouse.wheel(0, random.randint(100, 400))
        await asyncio.sleep(human_delay(0.3, 0.8, 1.5))
```

6.6 Proxy Rotation

Type	Cost	Detection Risk	Providers
Residential/ISP	$8–15/GB	Low	Oxylabs, Bright Data
SOCKS5 (VPN)	$3–5/month	Medium	NordVPN, Mullvad
Datacenter	$0.50–2/GB	High	Various

```
import random

proxies_list = [
    "http://user:pass@residential1.proxy.com:8080",
    "http://user:pass@residential2.proxy.com:8080",
]

def get_proxy():
    proxy = random.choice(proxies_list)
    return {"http": proxy, "https": proxy}
```

```
resp = requests.get(url, proxies=get_proxy())
```

Tip

Before a large scrape through a proxy, test it: check the IP (`httpbin.org/ip`), measure latency, and verify the target site responds normally. Dead proxies silently destroy throughput.

6.7 CAPTCHA Solving with Tesseract OCR

Government portals use simple image CAPTCHAs (distorted text on noisy backgrounds). The pipeline: download, remove noise, threshold, OCR, validate.

These are not reCAPTCHA. Simple image CAPTCHAs are solvable locally with OCR.

```
from PIL import Image
import pytesseract, requests
from io import BytesIO

def solve_captcha(image_url, session, noise_colors=None):
    """Download a CAPTCHA, clean it, OCR the text."""
    resp = session.get(image_url)
    img = Image.open(BytesIO(resp.content)).convert("RGB")
    pixels = img.load()

    # Remove noise: replace known noise colors with white
    if noise_colors:
        for y in range(img.height):
            for x in range(img.width):
                r, g, b = pixels[x, y]
                for nr, ng, nb in noise_colors:
                    if abs(r-nr)<20 and abs(g-ng)<20 and abs(b-nb)<20:
                        pixels[x, y] = (255, 255, 255)
                        break

    # Binary threshold
    gray = img.convert("L")
    binary = gray.point(lambda p: 255 if p > 140 else 0)

    # OCR with restricted character set
    config = ("--psm 7 -c tessedit_char_whitelist="
              "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789")
    return pytesseract.image_to_string(binary, config=config).strip()

def solve_with_retry(image_url, session, submit_fn, max_attempts=5):
```

```
    """Retry loop: each attempt is 60-80% accurate."""
    for attempt in range(max_attempts):
        text = solve_captcha(image_url, session,
                             noise_colors=[(200, 100, 100)])
        if len(text) < 4:
            continue
        result = submit_fn(text)
        if result.get("success"):
            return result
    return None  # Failed after max attempts
```

Individual OCR accuracy is 60–80%. With 5 retries, $(0.3)^5 \approx 0.2\%$ failure rate, giving >99% effective success.

6.8 CAPTCHA Avoidance Strategies

```
def test_captcha_enforcement(session, form_url, form_data):
    """Test if CAPTCHA is validated server-side."""
    # Submit with a nonsense value
    test_data = {**form_data, "captcha": "XXXXXX"}
    resp = session.post(form_url, data=test_data)
    if resp.status_code == 200 and "error" not in resp.text.lower():
        print("CAPTCHA is CLIENT-SIDE ONLY -- no solving needed!")
        return "client_side"

    # Submit without the field entirely
    test_data = {k: v for k, v in form_data.items() if k != "captcha"}
    resp = session.post(form_url, data=test_data)
    if resp.status_code == 200 and "error" not in resp.text.lower():
        print("CAPTCHA field not required by server.")
        return "not_required"

    return "enforced"
```

A surprising number of government portals implement CAPTCHAs only in the browser, with no server-side validation.

Other avoidance strategies: (1) **Mobile APIs** often skip CAPTCHA. (2) **Alternative entry points**: "advanced search" may lack CAPTCHA. (3) **Session persistence**: some sites show CAPTCHA only once per session.

6.9 SSL Certificate Issues

Government portals are notorious for SSL problems: certificates signed by government CAs not in your trust store.

India's NIC and CCA root certificates are not trusted by default in most operating systems.

Warning

Never use `verify=False` in production. It disables all transport security. Instead, download the specific CA certificate and pass it explicitly.

```
import certifi

def build_custom_ca_bundle(extra_cert_path, output="custom_ca.pem"):
    """Add a government CA cert to the default bundle."""
    with open(certifi.where()) as f:
        default = f.read()
    with open(extra_cert_path) as f:
        gov_cert = f.read()
    with open(output, "w") as f:
        f.write(default + "\n" + gov_cert)
    return output

session = requests.Session()
session.verify = build_custom_ca_bundle("india_cca_root.pem")
```

7 Building Robust Pipelines

7.1 Resumable Progress Tracking

This chapter turns a fragile script into production-grade infrastructure.

Every long-running scraper will be interrupted. Three approaches exist, each suited to a different scale.

7.1.1 Approach 1: JSON Progress File

JSON progress works for up to ~100K items. Beyond that, loading the file into memory on startup becomes slow.

```
import json
from pathlib import Path

PROGRESS_FILE = Path("output/progress.json")

def load_progress():
    if PROGRESS_FILE.exists():
        with open(PROGRESS_FILE) as f:
            return set(json.load(f))
    return set()

def save_progress(done: set):
    with open(PROGRESS_FILE, "w") as f:
```

```
        json.dump(sorted(done), f)

done = load_progress()
for item_id in all_ids:
    if item_id in done:
        continue
    data = scrape_item(item_id)
    save_item(item_id, data)
    done.add(item_id)
    save_progress(done)
```

7.1.2 Approach 2: SQLite with WAL Mode

```
import sqlite3

def init_db(db_path="output/progress.db"):
    conn = sqlite3.connect(db_path)
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("""CREATE TABLE IF NOT EXISTS progress
                    (item_id TEXT PRIMARY KEY,
                     scraped_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)""")
    conn.commit()
    return conn

def is_done(conn, item_id):
    return conn.execute("SELECT 1 FROM progress WHERE item_id=?",
                        (item_id,)).fetchone() is not None

def mark_done(conn, item_id):
    conn.execute("INSERT OR IGNORE INTO progress (item_id) VALUES (?)",
                 (item_id,))
    conn.commit()
```

WAL allows concurrent reads while one process writes, useful when monitoring progress while the scraper runs.

7.1.3 Approach 3: File-Existence Check

```
from pathlib import Path
OUTPUT_DIR = Path("output/items")

for item_id in all_ids:
    if (OUTPUT_DIR / f"{item_id}.json").exists():
        continue  # Already scraped
```

```
    data = scrape_item(item_id)
    save_atomic(OUTPUT_DIR / f"{item_id}.json", data)
```

Tip

Combine approaches for hierarchical scraping. Use file-existence checks for coarse progress (districts) and SQLite for fine-grained tracking (items within each district).

7.2 Atomic File Writes

If your script crashes mid-write, the output file exists but contains partial data. On restart, the progress tracker sees the file and skips it. You now have corrupt data that looks complete.

"Atomic" means the operation either completes fully or has no effect. No intermediate corrupted state.

```
import json, os, tempfile
from pathlib import Path

def save_atomic(path, data):
    """Write data to path atomically."""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp_path = tempfile.mkstemp(dir=path.parent, suffix=".tmp")
    try:
        with os.fdopen(fd, "w") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
        os.replace(tmp_path, path)  # Atomic on POSIX
    except:
        os.unlink(tmp_path)
        raise
```

Warning

The temporary file **must** be on the same filesystem as the final path. `os.replace()` is only atomic when source and destination share a filesystem.

7.3 Retry with Exponential Backoff

7.3.1 Automatic Retries with urllib3

The `Retry` adapter handles transient failures transparently inside the session.

```python
from urllib3.util.retry import Retry
from requests.adapters import HTTPAdapter

def make_session():
    session = requests.Session()
    retry = Retry(
        total=5, backoff_factor=1,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET", "POST"],
    )
    session.mount("https://", HTTPAdapter(max_retries=retry))
    session.mount("http://", HTTPAdapter(max_retries=retry))
    return session
```

7.3.2 Manual Retry Loop

```python
import time, random, requests

def fetch_with_retry(session, url, max_retries=5):
    for attempt in range(max_retries):
        try:
            resp = session.get(url, timeout=30)
            resp.raise_for_status()
            return resp
        except requests.exceptions.RequestException as e:
            if attempt == max_retries - 1:
                raise
            wait = (2 ** attempt) + random.uniform(0, 1)
            time.sleep(wait)
```

7.4 Circuit Breaker Pattern

Named after electrical circuit breakers: when failures exceed a threshold, the breaker trips open and stops the flow.

```python
import time

class CircuitBreaker:
    def __init__(self, failure_threshold=5, recovery_timeout=60):
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.failures = 0
        self.state = "closed"  # closed | open | half-open
        self.opened_at = None

    def allow_request(self):
        if self.state == "closed":
            return True
        if self.state == "open":
```

```
            if time.time() - self.opened_at >= self.recovery_timeout:
                self.state = "half-open"
                return True
            return False
        return self.state == "half-open"

    def record_success(self):
        self.failures = 0
        self.state = "closed"

    def record_failure(self):
        self.failures += 1
        if self.failures >= self.failure_threshold:
            self.state = "open"
            self.opened_at = time.time()
```

7.5 Session Refresh Strategies

Government portals are notorious for short session timeouts, sometimes as low as five minutes.

```
class SessionManager:
    def __init__(self, make_session_fn, max_requests=100):
        self.make_session = make_session_fn
        self.max_requests = max_requests
        self.request_count = 0
        self.session = self.make_session()

    def get(self, url, **kwargs):
        if self.request_count >= self.max_requests:
            self._refresh("request count limit")
        resp = self.session.get(url, **kwargs)
        self.request_count += 1
        if "login" in resp.url.lower():
            self._refresh("redirected to login")
            resp = self.session.get(url, **kwargs)
        return resp

    def _refresh(self, reason):
        print(f"Refreshing session: {reason}")
        self.session = self.make_session()
        self.request_count = 0
```

7.6 Data Storage Patterns

Scale	Format	Notes
<10K records	Single CSV or JSON	Simple, one-file output
10K–100K	Per-entity JSON + CSV	Individual files as source of truth
100K–1M	Chunked JSONL (5K/chunk)	Streaming writes, easy to resume
>1M	Sharded per-entity JSON	Shard by ID prefix (first 2 chars)
Spatial data	GeoPackage (.gpkg)	Single-file spatial database

Per-entity JSON files are the source of truth. Combined files are regenerable views.

```
import json, csv
from pathlib import Path

def build_combined(input_dir, output_csv):
    """Rebuild combined CSV from individual JSON files."""
    files = sorted(Path(input_dir).glob("*.json"))
    if not files:
        return
    with open(files[0]) as f:
        fieldnames = list(json.load(f).keys())
    with open(output_csv, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for path in files:
            with open(path) as jf:
                writer.writerow(json.load(jf))
    print(f"Combined {len(files)} records into {output_csv}")
```

7.7 Canonical Records with History Merging

When you re-run a scraper months later, merge new data with existing records. Deduplicate by natural key (receipt number, parcel ID).

```
def canonicalize(existing_dir, new_dir, output_dir, key_field):
    """Merge new scrape with existing data."""
    existing = {}
    for f in Path(existing_dir).glob("*.json"):
        record = json.loads(f.read_text())
        existing[record[key_field]] = record

    new_count, updated = 0, 0
```

```
    for f in Path(new_dir).glob("*.json"):
        record = json.loads(f.read_text())
        key = record[key_field]
        if key in existing:
            existing[key].update(record)
            updated += 1
        else:
            existing[key] = record
            new_count += 1

    Path(output_dir).mkdir(parents=True, exist_ok=True)
    for key, record in existing.items():
        save_atomic(Path(output_dir) / f"{key}.json", record)
    print(f"{new_count} new, {updated} updated, {len(existing)} total")
```

Tip

Always use the natural key, not filenames. Receipt numbers, parcel IDs, or license numbers are stable identifiers. Filenames and URLs can change between runs.

8 Putting It All Together

8.1 The Standard Pipeline Template

This template is a starting point. Remove what you do not need, but think twice before removing progress tracking or atomic writes.

```
"""Standard scraper template. Copy and adapt for each project."""
import argparse, json, os, random, sqlite3, tempfile, time
import requests
from pathlib import Path
from urllib3.util.retry import Retry
from requests.adapters import HTTPAdapter

# --- Config ---
BASE_URL = "https://example.gov/api"
OUTPUT_DIR = Path("output/items")
DB_PATH = "output/progress.db"
DELAY_RANGE = (1.0, 3.0)

# --- Session factory ---
def make_session():
    session = requests.Session()
    retry = Retry(total=5, backoff_factor=1,
                  status_forcelist=[429, 500, 502, 503, 504])
```

```
    session.mount("https://", HTTPAdapter(max_retries=retry))
    session.headers.update({"User-Agent": "ResearchBot/1.0"})
    return session

# --- Progress tracking (SQLite) ---
def init_db():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("PRAGMA journal_mode=WAL")
    conn.execute("""CREATE TABLE IF NOT EXISTS progress
                    (item_id TEXT PRIMARY KEY,
                     scraped_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)""")
    conn.commit()
    return conn

def is_done(conn, item_id):
    return conn.execute("SELECT 1 FROM progress WHERE item_id=?",
                        (item_id,)).fetchone() is not None

def mark_done(conn, item_id):
    conn.execute("INSERT OR IGNORE INTO progress (item_id) VALUES (?)",
                 (item_id,))
    conn.commit()

# --- Atomic writes ---
def save_atomic(path, data):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=path.parent, suffix=".tmp")
    try:
        with os.fdopen(fd, "w") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
        os.replace(tmp, path)
    except:
        os.unlink(tmp)
        raise

# --- Main loop ---
def scrape(item_ids, session, conn):
    for i, item_id in enumerate(item_ids):
        if is_done(conn, item_id):
            continue
        try:
            resp = session.get(f"{BASE_URL}/{item_id}", timeout=30)
```

```
            resp.raise_for_status()
            save_atomic(OUTPUT_DIR / f"{item_id}.json", resp.json())
            mark_done(conn, item_id)
            if (i + 1) % 100 == 0:
                print(f"Progress: {i+1}/{len(item_ids)}")
        except requests.exceptions.RequestException as e:
            print(f"Failed {item_id}: {e}")
        time.sleep(random.uniform(*DELAY_RANGE))

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", required=True)
    parser.add_argument("--limit", type=int, default=0)
    args = parser.parse_args()
    with open(args.input) as f:
        item_ids = json.load(f)
    if args.limit:
        item_ids = item_ids[:args.limit]
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    scrape(item_ids, make_session(), init_db())
```

8.2 Checklist for Starting a New Scraping Project

Tip

New project checklist:

- ☐ Identify the data source and target fields
- ☐ Check for APIs (DevTools Network tab, look for JSON)
- ☐ Check `robots.txt` and Terms of Service
- ☐ Determine tech stack (ASP.NET? JS-rendered? REST API?)
- ☐ Choose tools: `requests` for APIs, `Playwright` for JS, `aiohttp` for high concurrency
- ☐ Plan progress tracking: JSON for <100K, SQLite for larger jobs
- ☐ Plan output: JSON per entity (source of truth) + CSV combined view
- ☐ Set up atomic writes from the start
- ☐ Add retry logic with exponential backoff
- ☐ Test with a small sample (`-limit 10`) before full scale

8.3 Common Mistakes and Debugging

Every row represents a lesson learned the hard way.

Mistake	Fix
Writing directly to final path	Use atomic writes
No resume support	Add progress tracking first
Fixed delays between requests	Randomize with `random.uniform()`
Catching bare `Exception`	Catch specific exceptions
Storing all data in memory	Write incrementally to disk
Hardcoding URLs and selectors	Use constants; extract dynamically
Using `verify=False`	Embed the missing CA cert
No logging	Use `print` with timestamps at minimum

8.4 Debugging Strategies

Empty results: Check whether the page structure changed. Fetch one page with `requests.get()` and inspect the HTML. If the site switched to JavaScript rendering, switch to Playwright.

Getting blocked (403): Check request rate. Examine headers. Add longer delays. Rotate User-Agent strings. If the site uses bot detection, check whether you need session cookies.

Incomplete data: The page layout varies across records. Use `.get()` or `.select_one()` with fallback values instead of assuming every field exists.

Tip

The debugging decision tree:

1. Does the request reach the server? Check for `ConnectionError`.
2. Does the server respond with 200? If not, check the status code.
3. Does the response contain expected HTML/JSON? Print `resp.text[:500]`.
4. Do your selectors match? Test against the manually fetched HTML.

8.5 Resources for Further Learning

Python libraries:

- `requests` – HTTP for humans. Default choice for API scraping.
- `beautifulsoup4` – HTML/XML parser. Pair with `requests`.
- `lxml` – Fast parser backend. Supports XPath.
- `playwright` – Browser automation for JS-rendered pages.
- `aiohttp` – Async HTTP client for high-concurrency scraping.
- `scrapy` – Full framework with built-in concurrency and pipelines.

Reference documentation:

- Python requests: https://docs.python-requests.org
- BeautifulSoup: https://www.crummy.com/software/BeautifulSoup/
- Playwright for Python: https://playwright.dev/python/
- SQLite in Python: https://docs.python.org/3/library/sqlite3.html
- robots.txt specification: RFC 9309